AirOS 路侧感知系统 — 面试 QA 精简版（35 题）

> **这份是什么** 从 `0面试QA总表.md`（129 题）里挑出的**最核心 35 题**，用来考前突击、临场前快速过一遍。**题号沿用原表**（看到 Q2-3 就是原表的 Q2-3），想深挖某一题，回原表按题号查即可。 **不替代原表**——原表是字典，这份是常用字表。
>
> **挑选标准**：`[必背]` + `★★` 高频 + **诚实题**（容易穿帮的）+ 每个条目的**总览题**。 砍掉的是同类重复题和纯加分储备题——**不是说不考，是优先级低。**
>
> **怎么用**
>
> 1. **每个条目的第 1 题是"目录"**：答案里抛出的每个词，下面都有编号追问，按真实面试被问的顺序铺好了。
> 2. 其余题保留 1 条主追问，用来接住"为什么 / 会怎样"。
> 3. 答案都能**直接念出口**，念完 20~30 秒；念超时说明你加戏了。
>
> 🔴 **四条全局红线（全文通用，先看完再背）**
>
> 1. 全程说「**参与**」，不说「我从零设计」。
> 2. 视频那条说「**拉流和拆包**」，**绝不说解码**——本驱动没有解码环节。
> 3. 部署工具写「独立开发并维护升级」；**口语里主动说用了 AI 辅助**，别等被问出来。
> 4. 交通参数检测那条**另有专属红线**，写在该条开头，**先看完再背题。**
>
> ⏱️ **突击顺序建议**：开场 3 题 → 附录角色题 → 条目二 → 条目四 → 条目一 → 条目三。

目录

- 第 0 部分：开场自述（3 题）
- 条目一：设备抽象与插件化接入（6 题）
- 条目二：多路视频流接入（8 题）
- 条目三：设备与平台运维（5 题）
- 条目四：交通参数检测服务（9 题）
- 附录：通用追问（4 题）

第 0 部分：开场自述（3 题）

Q0.1 简单介绍一下你这个项目是干什么的? [必背]

答： 我参与的是一个车路协同的**路侧系统**，跑在路口的边缘盒子上，干的事就是把相机、雷达、信号机这些设备接进来，数据收上来再发给平台和车端。C++ 写的，跑在容器里，硬件是带 GPU 的嵌入式板子。 我主要做了两块。**设备接入**这块：设备的**插件化接入**、**多路视频流**的拉取和链路调试、边缘设备的**运维**（GPU 指标采集，加上一个给现场用的批量部署工具）。**交通参数检测**这块：订阅上游融合感知输出的目标流，算路口的**流量、排队、车速、车头时距**这些指标，包括检测器框架、数据管道和车辆过线判定。

追问 1：边缘盒子是什么？为什么算力要放在路口，不上云?

> 就是放在路口机柜里的一台小工控机，带 GPU。必须在本地算是因为**延时和带宽**——视频流全量回传带宽扛不住，而车路协同要的是毫秒级响应，绕一圈云端就来不及了。云只用来做管理和长期存储。

追问 2：你说的"接进来"，具体接了哪几类设备?

> 主要是**相机、雷达、信号机**这几类。相机出视频流，雷达出目标点云，信号机给的是路口的灯态相位。它们协议完全不一样，所以才需要那套统一接入接口把它们抹平。

追问 3：为什么用 C++？又为什么跑在容器里?

> C++ 是因为**要压榨边缘设备的算力**，而且要在多线程下稳定跑几个月不重启。跑容器是为了**部署和环境隔离**——几十个路口的硬件批次不同，容器能保证我这份代码在任何一台上跑起来都一样。

追问 4：你们这个系统是分层设计的吗?

> 是。上面应用层、中间服务层、下面硬件抽象层。分层的好处是：加一个新设备不用动上面的代码，按标准实现一个"插件"挂上去就行。

追问 5：「插件化接入」具体是什么意思?

> 驱动编译成独立的动态库，框架启动时扫目录加载，驱动里一行注册宏把自己登记进工厂。所以加一款新设备是**加一个文件 + 改一行配置**，框架代码不用动。 👉 深挖见**条目一 Q1-1**。

追问 6：多路视频流是怎么组织的?

> **一路流 = 一份私有状态 = 一个独立线程**，一个设备管理多路。路跟路之间互不干扰，一路断了不影响别的路。 👉 深挖见**条目二 Q1-1**。

追问 7：运维具体运维什么？部署工具是干嘛的?

> 两块：一是**采 GPU 利用率、显存、温度**这些指标上报，看设备健不健康；二是我独立做的一个**桌面批量部署工具**，把"运维拿 Excel 一台一台 SSH 敲命令"变成"选表 + 点按钮"。 👉 深挖见**条目三 Q1-1 / Q3-1**。

追问 8：交通参数那块，你订阅的到底是什么数据?

> 订阅的是**上游融合感知输出的结构化目标流**——一帧里所有目标的清单，每条带 id、车型、经纬度、速度、车道号。**我从头到尾没见过像素**，解码和检测跟踪是算法团队做的。 👉 深挖见**条目四 Q1-1**。

追问 9：你这两块之间是什么关系？

> 是同一条业务链的**首尾两端**。前面我负责把设备数据接进来，中间解码、目标检测跟踪、多传感器融合是算法团队做的，他们产出结构化结果后我才在后面算交通参数。**中间那一棒不是我做的**。

> 📍 **这一答抛出了整张地图**——被追问时往哪儿走：
>
> - **"插件化接入"** → 条目一 Q1-1
> - **"多路视频流"** → 条目二 Q1-1
> - **"运维 / GPU 指标采集"** → 条目三 Q1-1
> - **"部署工具"** → 条目三 Q3-1
> - **"交通参数检测 / 订阅目标流"** → 条目四 Q1-1
> - **"两块什么关系 / 哪些不是你做的"** → Q0.3、附录 Q1
>
> 背法：Q0.1 是**全表目录**。开场这 40 秒答完，面试官接下来问的每一句，基本都落在上面这张表里。

Q0.2 你说的「框架 + 组件」到底是什么意思？ [必背] ★ 高频

答： 框架是**插座**，组件是**插头**。框架只规定"插头必须长什么样"，不管你插的是台灯还是风扇。所以以后加新设备、加新算法，框架代码一个字都不用改。 具体到相机：框架负责扫描加载驱动、创建实例、把数据发出去；驱动负责按标准实现某款设备。两层中间靠一套**抽象接口**衔接。

追问：这个设计有什么不好？

> 两个。一是接口一旦定死，扩展就要动所有实现，所以定接口得先想清楚粒度；二是加载失败报错不直接，现象是"配置里写了这个名字但框架说找不到"，得翻日志定位。这也是我想改进的地方。

Q0.3 你做的这些，和「交通参数检测」是一回事吗？ [必背] ★★ 开场必被问

答： 两块都属于公司的路侧产品线，装在同一台边缘设备上，是同一条业务链的**首尾两端**：我前面负责把设备数据接进来、保证链路稳，后面负责订阅上游的感知结果算流量、排队这些指标。 中间那一棒**不是我做的**——数据接进来要解码、做目标检测跟踪、多传感器融合，那是算法团队的活。他们给我的是**结构化结果**：一条条参与者记录，带类型、经纬度、速度、车道号。

> 🔴 **红线**：绝不能说"我拉的视频流直接给了流量模块"。中间隔着解码和感知融合，一追问就穿帮。

追问 1：那你拉的数据到底发给谁了？

> 发到框架的**消息总线**上了，谁订阅谁收到。具体谁是订阅方属于感知融合那一块，是算法团队的模块，我没参与。我只保证这一棒是稳的——能连上、断了能重连、出问题能从指标上看出来。

追问 2：那为什么两边用的通信方式不一样？

> 不同团队的选型。主框架用一套通信中间件加 protobuf；交通参数检测那边的形态是**算法插件**，跑在一个宿主程序里，宿主给它的是 C 接口加另一套通信库，数据格式是 JSON。两边业务上靠**标准化的感知消息**衔接，通信各走各的。

追问 3：所以到底哪些是你做的？

> 我做的：多路视频流的拉取、设备的插件化接入、GPU 指标采集、部署工具，还有交通参数那边的检测器框架、过线判定、生产消费管道。**没做的**：图像解码、目标检测和跟踪、多传感器融合。

📌 ***自己复习：*** `6两个模块-合成一个故事.md` ***+ 图*** `imgs/07-两个模块怎么串成一条链路.svg`

条目一：设备抽象与插件化接入（6 题）

简历原话 设备抽象与插件化接入：为相机等路侧设备定义统一抽象基类接口（初始化、启停、数据下发、状态查询），各厂商驱动通过虚函数重写实现同一套接口；配合工厂模式 + 注册宏完成驱动自注册与动态库加载，新增设备型号只需新增驱动文件、框架代码零改动，接入方式从"改代码"变为"改配置"。

🔴 **本条两条红线**

1. 全程说「**参与**」——接口和工厂是 mentor 定的架构，我负责具体模块落地。
2. "框架代码零改动"指的是**新增一款设备**不用改框架；**新增一类能力还是要动接口**，被追问要能说清这个边界。

❗ **"统一"是按设备类型各一套**（相机 / 雷达 / RSU / 信号灯……各一份接口 + 工厂），**没有所有设备共用的公共基类。** ❗ **别把"厂商"说成"厂商 SDK"**：我做的是**通用 RTSP + FFmpeg** 协议层接入，**没对接任何厂商私有 SDK。**

技术点 1：统一接口与多态（1 题）

Q1-1 你说的「统一接入接口」，具体是怎么设计的? [必背]

答： 就三步：先定一套**接口**，再让每家厂商的驱动各自去实现，最后框架只认这套接口。接口上有四个方法——**初始化、启动采集、查状态、下发指令**；另外留了一条**回调**给数据上行，设备采到数据就交给它，交给谁由框架定。 能统一成这四个方法，是因为不管哪家设备，一生的动作都一样：先配置资源、再开始出数据、中间要能查健康、有的还要能反向下发指令。只是每一步具体怎么做各家不同。统一之后框架就不用管背后是谁了。



橙色回环 = 数据上行：驱动只负责把数据交给回调，不关心发给谁

> 看图要点：从上往下是「框架 → 工厂 → 抽象基类 → 具体驱动」，橙色回环是数据上行。上面三层只认**抽象基类**这一个类型，最底下那个驱动是后加的，框架一行都不用改。

> ❗ **别只答"虚函数基类 + 子类重写"**——那只讲了多态，没讲插件化。完整的是**三件套**：
>
> 1. **抽象基类**（纯虚函数）——定能力清单；
> 2. **工厂 + 注册宏**——驱动文件末尾一行注册宏，把"名字 → 构造函数"登记进表，**发生在 main 之前**；
> 3. **动态库加载**——驱动编译成独立 `.so`，启动时扫目录加载。
>
> ❗ **"统一"是按设备类型各一套**（相机、雷达、RSU、信号灯……各一份接口 + 工厂），**没有所有设备共用的公共基类**。被追问就诚实说：按类型分别统一，抽公共基类是后续优化点。
>
> ❗ **别把"厂商"说成"厂商 SDK"**：我做的是**通用 RTSP + FFmpeg** 的协议层接入，一套代码通吃各家，**没对接任何厂商私有 SDK**。

追问 1：那这个接口在代码里是怎么落地的？

> 就是一个**抽象基类**，四个方法全是**纯虚函数**——基类只声明不实现，谁继承谁就必须自己写。框架手里拿的是基类指针，调的时候自动跑到具体那份实现上，这就是运行时多态。

追问 2：为什么正好是这四个方法？不能多不能少？

> 因为不管哪家设备，**一生的动作都一样**：先配置资源、再开始出数据、中间要能查健康、有的还要能反向下发指令。这四个覆盖了设备全生命周期，差别只在每一步怎么做。 ⚠️ 真遇到某家不支持其中某个，让它**空实现返回不支持**就行，不能为了迁就它去改接口——改接口会波及所有已接入的驱动。

追问 3：为什么用纯虚函数？普通虚函数不行吗？

> 不行。普通虚函数意味着基类**给了默认实现**，子类不写也能编译过——运行时就变成一个"什么都没干"的静默失败，现场查起来极其痛苦。纯虚函数把这件事**逼到编译期**：漏实现就编译不过。 而且基类本来也给不出有意义的默认实现——一台相机怎么初始化，基类怎么会知道。

追问 4：用基类指针销毁对象，要注意什么？

> **基类析构函数必须是虚函数。** 否则通过基类指针 delete 的时候，只会调到基类的析构，子类那部分资源就泄漏了——而且这种泄漏是静默的，跑几天才看出来。 这也是我判断一个"面向接口"的设计成不成熟时最先看的一点：**只要打算用基类指针持有对象，析构就必须虚。**

追问 5：那条回调为什么选择在构造函数里传进来？

> 这叫**构造期注入**。最大的好处是：回调变成对象一旦存在就**必然可用**，不会出现"建好了但还没设回调"这种半初始化状态。如果做成 setter，就多了一个必须记得调用的步骤，忘了就静默丢数据。

追问 6：这套接口最大的问题是什么？重做你会怎么改？

> 两个。一是**接口一旦定死，扩展就要动所有实现**，所以定接口得先想清楚粒度；二是**各类设备各有一套接口，没有公共基类**，同构代码重复了九份。重做的话我会先抽一层真正公共的基类，再往下按类型分化。

✅ **一句话版本（面试照背）**

它的核心确实是抽象基类和虚函数重写——接口层把能力定成纯虚函数，驱动继承下来各自实现，框架只拿基类指针调。但光有这层还不叫插件化：我在接口之外还配了**注册工厂 + 动态库加载**，驱动里一行注册宏把自己登记进工厂，框架侧只认配置里的一个名字。所以新增一个型号，是加一个驱动文件 + 一行注册，框架代码一行都不动——这才是我说的「统一接入」。

📌 **自己复习（面试别说）：**

- 接口：`base/device_connect/ipcamera/device_base.h` 的 `IpCameraDevice`，四个纯虚 `Init / Start / GetState / WriteToDevice`，`protected: IpCameraCallBack sender_`，回调由构造函数注入。
- 子类：`standard_ipcamera.h:44` `StandardIpCamera : public IpCameraDevice`，四方法全 override。
- 工厂：`device_factory.h` 单例 + `map<string, CONSTRUCT>`；宏 `V2XOS_IPCAMERA_REG_FACTORY(StandardIpCamera, "standard_ipcamera")`，注册点 `standard_ipcamera.cc:608`。
- 框架取实例：`ipcamera_component.cc:42` `GetUnique(conf_.device(), ...)`；同文件 :31 动态加载 `device/lib/ipcamera/`。
- 同构共 9 套（camera / ipcamera / lidar / radar / rsu / traffic_light / mec / ins / cloud），每类一份 device_base.h + device_factory.h。
- 老的 `modules/camera/ipcamera/include/camera.h`（按 `Vendor` 分海康/大华/仿真）走厂商 SDK 路线，跟 standard_ipcamera 不是一条路，别混讲。

技术点 2：工厂 + 自注册（3 题）

Q2-1 为什么要用工厂模式？直接在框架里判断类型创建对象不行吗？[必背] ★ 高频

答：在框架里写判断的话，每来一个新厂商我就要打开框架代码改一遍、再把整个框架重编一遍。框架是所有人共用的基础件，张三接入改一次、李四接入改一次，风险和协调成本都很高。还有个更隐蔽的问题：这么写之后框架就**依赖了所有具体驱动**，随便哪个驱动改个头文件，框架都得重编。 工厂把"选哪个设备"这个决定，**从编译期推迟到了运行期，从代码里搬到了配置文件里**。框架只认那套抽象接口，具体类型全藏在注册表后面。

追问：那跟"简单工厂"比，你这个好在哪？

> 简单工厂只是把那堆判断挪了个地方，工厂自己还是知道世界上有哪些设备。我们这个是**注册表由各个驱动自己注册进来，工厂完全不知道有哪些设备存在**。所以新增设备的代价从"改工厂"降到了"新写一个文件"，这是本质区别。

Q2-3 你说的「一行注册宏」，它到底是怎么工作的？[必背] ★★ 高频

答：每个驱动文件末尾写一行宏，这行宏展开之后其实是定义了一个**静态对象**，它的构造函数干的事就是：把自己的设备名和"怎么造自己"登记到工厂的注册表里。 关键就在"静态"两个字——C++ 保证静态对象的构造函数会在程序加载的时候自动执行，不需要谁手动去调。所以框架把驱动加载进来，注册就自动完成了，各驱动完全自己管自己，不需要在任何地方维护一份"设备清单"。

追问：但是静态对象的初始化顺序是不确定的，会不会出问题？

> 这个担心是对的，C++ 确实不保证不同源文件之间静态对象的初始化顺序。但这里恰好安全，关键在于**工厂是用函数内的局部静态变量实现的单例**——用到的时候才构造，所以不存在"注册跑到一半、工厂还没建好"的情况。反过来，如果工厂是个文件作用域的全局变量，注册可能在它构造之前发生，往一个还没初始化的容器里塞东西就是未定义行为。 所以准确说法不是"静态对象顺序一定安全"，而是**用局部静态单例把它变成了按需构造**才安全。

追问 1：静态对象到底什么时候构造？动态库呢？

> 可执行文件里的静态对象，在 **main 之前**就构造好了；动态库里的是**加载时才构造**——也就是框架扫描目录、把 `.so` 加载进来的那一刻。所以注册这个动作发生在框架真正开始工作之前，等它去查表时表里已经有东西了。

追问 2：宏里面要用符号拼接生成变量名，为什么？

> 因为宏在**每个驱动文件里都要用一次**，如果不拼接就会生成同名的全局变量，链接期直接报重复定义。拼接上类名之后，每个驱动拿到的是自己那份独一无二的变量名。

追问 3：工厂是单例，多线程同时调用安全吗？

> 要分两段看。**注册**发生在加载期，那时候还没起工作线程，天然串行；**查询创建**在运行期，多线程可能同时进来。所以查表那一步要加锁保护——这也是为什么我倾向于让注册越早完成越好，运行期只做只读查询。

追问 4：如果两个驱动注册了同一个名字会怎样？

> 后加载的那个会把先来的**覆盖掉**，运行时的现象是"配了这个名字但行为不对"，而且不报错、很难查。所以更稳妥的做法是**重复注册时直接报错退出**，而不是静默覆盖。

追问 5：宏用起来爽，代价是什么？

> 调试很痛苦——宏展开后的代码和看到的不一样，报错信息也难读。而且它是**编译期**的黑魔法，出了问题只能看预处理后的文件排查。这是我认为这套设计里最该被替换掉的部分，更好的做法是改成显式注册函数。

Q2-8 用具体流程说说：新来一款设备，你要做哪些事？ [必背] ★★ 高频场景题

答： 四步，**全程不碰框架代码。**第一，新建一个驱动文件，写一个类继承那套统一接口，把那几个方法实现了；第二，文件末尾加一行注册宏，名字填厂商加型号；第三，把这个模块编译成独立的动态库，放到框架会去扫描的那个目录下；第四，改配置，把里面的设备名从老驱动改成新的。 重启之后框架扫描目录加载起来，注册自动完成，就按新配置创建实例了。**框架代码零改动，也不需要重新编译框架本身。**

追问：这套流程的瓶颈在哪？

> 主要两个：一是接口是有限的，遇到厂商有特别能力的时候要扩展统一接口；二是出问题得靠日志排查，没有已注册清单的时候比较绕。这两个都是我前面说的改进点。

技术点 3：动态加载（1 题）

Q3-2 加载失败会有什么现象？你怎么排查？ [加分] ★ 有实战感

答： 现象是**设备名查不到**——配置里明明写了这个名字，但创建实例的时候失败，打一条错误日志。 麻烦的是失败原因不会直接暴露，要用专门的错误查询函数才能拿到。所以我一般三步：用查依赖的工具看这个库是不是缺了某个依赖；确认那个依赖在搜索路径里能找到；把依赖补上，或者在加载目标库之前**先显式地把依赖加载进来**。

追问：你实际遇到过吗？

> 遇到过。有次一个动态库始终加载失败，现象就是它的设备注册不上。后来发现它依赖了某个第三方库，而那个库当时还没被加载。解决办法就是加载驱动之前先检查依赖在不在，不在就按顺序先加载依赖。这类问题编译期完全发现不了，只能靠启动日志。

技术点 4：改配置不改代码（1 题）

Q4-1 你说的「改配置」，具体改的是哪几层？ [必背] ★

答： 一共三层，分工很明确。第一层是**进程编排文件**，它决定这个进程要加载哪个动态库、实例化哪个类、配置文件在哪；第二层是**组件配置**，里面最关键的一个字段就是**指定用哪个驱动**，以及指向下一层配置文件的路径；第三层是**驱动自己的业务参数**，比如这台设备要连的几路视频流地址、每路输出到哪个通道、多久重连一次。 一句话概括：**第一层决定跑什么进程，第二层决定用哪个驱动，第三层决定这台设备具体怎么连。**

条目二：多路视频流接入（8 题）

> **简历原话** 多路视频流接入：参与多路 RTSP 视频流的拉取与链路调试，每路流由独立线程承载，基于 FFmpeg 完成拉流与拆包并封装为 protobuf 消息统一发布；支持断线自动重连（外层重连、内层读帧的双层循环）与帧间隔统计，便于现场定位链路抖动。
>
> 🔴 **本条两条红线（最容易穿帮）**
>
> 1. **绝不说"解码"**——代码里**没有解码环节**，只说拉流 + 拆包 + 封装。
> 2. **绝不说"我拉的流直接给了流量模块"**——中间隔着解码 + 目标检测跟踪 + 融合，那是算法团队的活。
>
> ⚠️ **背诵优先级**：真正要背的只有三句——① 这模块干嘛的 ② 核心实现（每路一个线程 + 双层循环）③ 我具体参与了什么。

技术点 1：每路流一个线程（1 题）

Q1-1 多路视频流，你整体上是怎么组织的？ [必背]

答： 一个设备管理多路流。内部有一张表，**键是这路的输出通道名**，值是这路的全部私有状态——视频流地址、连接上下文、它自己的线程、停止标志、连接状态、统计数据。 也就是：**一路流 = 一份私有状态 = 一个独立线程**，彼此之间只有那张总表是共享的。好处是每路自成一体，加一路减一路只影响表里的一项。

追问 1：为什么不让每路流各做一个独立对象，非要一张表？

> 因为框架要能**统一调度**——启动、停止、查状态这些动作是批量下发的。做成一张表，主循环 for 一遍就能让所有路一起动；各做独立对象的话，谁来统一管它们又成了新问题。

追问 2：那张表的"键"为什么是输出通道名，不是流的地址？

> 因为**地址可能会变**（现场改 IP、换相机），而输出通道名是**下游订阅时认的那个名字**，是稳定的契约。用地址当键的话，改一次 IP 下游的订阅关系就全断了。 ⚠️ 地址和通道是**配置里按位置一一对应**的：第一个地址对第一个通道。写错了不会报错，现象是"这路数据跑到了别的通道上"——这是现场最难查的错之一。

追问 3：为什么每路一个线程？用线程池不行吗？

> 不行，因为每路流的**生命周期和阻塞点是独立的**。拉流是长时间阻塞在 I/O 上的，如果塞进线程池，一条流卡住就占掉一个线程，路数一多池子就满了，后面的流根本轮不上。每路一个线程的好处是**谁断了、谁卡了都很清楚**，定位问题时能直接对到某一路。

追问 4：那份私有状态为什么用智能指针装着，不直接放进容器？

> 因为**线程要长期持有它**。直接放对象进去，容器内部调整结构的时候对象地址会变，线程那边抓着的引用就失效了。用引用计数的智能指针可以保证地址稳定，最后一处用完才释放。

追问 5：那停止的时候，线程是怎么退出来的？

> 不是强杀，是**置停止标志 + 让循环自己退出**。线程跑着双层循环，每轮都看一眼标志，看到就跳出、把连接关掉、正常结束。强杀线程会留下没关的连接和没释放的资源。

追问 6：这个每路一线程的模型，最大的缺点是什么？

> **路数一多，线程数就线性涨**。现场十几路还能扛，真到几十路，光线程栈和上下文切换就是一笔开销。要改的话得换成**少量线程 + 非阻塞 I/O** 的模型，但那样代码的复杂度会明显上升——当前规模下不划算，所以维持现状。

技术点 2：视频数据的拉取和拆分（4 题）

Q2-1 你用视频处理库，到底做了哪些事？哪些没做？ [必背] ★★ 最关键的一题

答： 我做的是**拉流和拆包**，**没有做解码**这一步。 拉流就是连上地址、协商参数、把数据接下来；拆包就是把连续的码流切成一个个压缩包。整个模块里**没有任何解码相关的调用**，读到的是**压缩的裸码流**，我原样封装发出去，**谁需要像素自己去解**。 好处是驱动这层职责最轻，也不替下游决定解码策略——下游可能走 GPU 硬解，也可能批量解，那是它的自由。

> 🔴 **红线**：简历写的是"拉流与拆包"，**不是"解码"**。被追问"你是不是做了解码"时直接答：我做的是拉流和拆包，像素级解压交给下游。

> 📌 *自己复习：另有一套 camera/ipcamera 目录的驱动用 CUDA 硬解，**那是另一套，面试别混说。***

Q2-3 建立连接那一步，是把整个视频流都拉下来了吗？ [必背] ★★ 最常答错

答： **不是。** 监控相机的视频是**持续不断的实时流**，不像下载文件那样有终点，压根没有"整个"这个概念。 建立连接只做三件小事：**握手建立连接、读一小段数据探测格式、把流的上下文结构体填好**。真正一包一包读数据的是第三步那个读帧操作，它在内层循环里被调用，跑一小时就是几十万次。

Q2-5 那一个包就等于完整的一帧图像吗？ [必背] ★★ 容易答错

答： **不一定**，两个方向都有偏差。 一个是**一帧会被切成好几个包**：比较大的帧，尤其是关键帧，要几个包才能拼成一帧。另一个是**有的包压根不是图像**：参数集、补充信息这些元数据也是以包的形式过来的。 所以简历上写"帧间隔统计"其实是**通俗说法**，严谨讲应该是"相邻两个视频包的间隔"。这个区分值得主动说出来。

Q2-6 读完之后的那个归还操作，是释放这个包吗？ [必背] ★★ 容易答错

答： **不是释放包，是减它手里那块数据的引用计数。** 包本身是**栈上的局部变量**，循环里每一轮复用同一个；真正要归还的是它内部指向的那块**数据缓冲区**——库内部有个引用池，这个操作把计数减一，减到零那块内存才真的回到池子。所以初始化那个包只需在进循环前做一次，后面就是反复"读 + 归还"。 为什么重要：一路三十

帧跑一小时就是十万次以上，漏一次就泄漏一块数据。我们开发初期踩过内存持续增长的坑，最后就是在**错误分支上漏了归还。**

> 📌 *自己复习：这跟 `free` 完全不同，是引用计数语义。*

技术点 3：数据封装与发布（2 题）

Q3-5 往外上报的时候，为什么必须把输出通道名一起传出去？ [必背] ★★ 核心设计点

答： 因为**驱动手里没有发送器，它不知道该往哪儿发。**驱动只知道"我是第几路，我刚拿到了第几个包"；而"第几路该发到哪个通道"是**框架才有的信息**（配置里配的）。 所以回调设计成两个参数：**数据本身，加这路的通道名。**打个比方，数据就是包裹，通道名就是快递单上的收件地址——我打包 + 写地址，快递公司负责投递。**不带地址，快递公司不知道往哪送。** 顺带还有个好处：总表的索引本来就是通道名，将来能直接反查回这路流的状态。

Q3-6 那数据最终是发给谁了？发给某个确定的模块吗？ [必背] ★

答： 这一点要诚实：**不是发给某个确定的模块，是发到框架的消息总线上了。** 一路流一个通道，这是**发布-订阅模型**：发布方不需要知道谁在订阅，谁订阅谁收到，**一个订阅者都没有也不报错。**正因为驱动不认识下游，它才必须把通道名交给框架去发；好处是下游增加、删除、换一拨人，驱动一行代码都不用改。 我能看到的订阅方主要是两个工具：一个把通道里的数据再转推成视频流方便联调，另一个从离线录制包里把视频数据抽出来存文件方便复现问题。**真正的业务下游在公司内部的业务代码里，开源仓库里看不到。** 这句话本身就是既诚实又专业的答法。

> 📌 *自己复习：仓库里另有一套同名前缀的相机驱动，但它用的是另一套消息定义、是独立的 SDK，**不是我们的下游**。同名只是约定了通道名，**同名不等于同一条链路**。*

技术点 4：断线重连（1 题）

Q4-1 你说的「双层循环」，具体是怎么配合的？ [必背] ★★ 核心

答： **外层管连接，内层管读数据。** 外层每次进来先试着连一下，连不上就等几秒然后回到外层顶部重新试；连上了就进入内层，内层只要还连着就一直不停地读包。读到一半失败了，就把连接状态置掉、跳出内层，出来之后把这条连接关掉，再回到外层重新开始连。 一句话概括：**外层负责"连不上就重试"，内层负责"连着就一直读"，读断了掉回外层重新连。** 好处是把"连接"和"读数据"这两种完全不同的关注点彻底分开，各自的状态都很简单。

追问 1：外层循环判断的是连接状态吗？

> 不是，它判断的是**循环自己的退出条件**。真正标记"连没连上"的是另一个独立的状态位，由内层在断流时置掉。把这两件事分开，外层就不用关心连接是怎么断的，只看要不要继续重连。

追问 2：连不上就睡几秒再重试，这算轮询吗？会不会占 CPU？

> 不算，这是**带退避的重试**，不是轮询。轮询是转着圈不停检查、CPU 一直耗着；我这个是**真的睡过去**，不占 CPU。而且睡一段时间还能避免设备刚重启时被疯狂连——那是把本来能恢复的设备打得更死。

追问 3：睡眠那句外面还包着一个判断，你知道吗？

> 知道。包的是**停止标志**——睡眠前先看一眼要不要停，要停就立刻退出，不再继续睡。不这么做的话，点了停止还得等这几秒睡完才响应，现场会觉得"点了没反应"。

追问 4：内层出来之后关的是什么？线程也关了吗？

> 关的是**这条连接和它的上下文**，线程**不关**。线程还活着，它会回到外层顶部继续尝试重连——所以这整个双层循环是在同一个线程里跑的，线程只在设备整体停止时才退出。

追问 5：读数据返回失败，就一定代表网络断了吗？

> **不一定**，这是最容易答错的一点。返回失败可能是这一包坏了、可能是暂时没数据、也可能是真的断了。我这里的处理是**一律当作断了跳回外层重连**——因为区分这些的成本很高，而重连的代价很低。这是**有意的取舍**，不是没考虑到。

追问 6：一直连不上会怎样？会不会退出？

> **不会退出，会一直重试下去**，直到设备整体被停止。这是边缘设备的基本素养——现场网络抖动、相机重启都很常见，模块自己先放弃的话，网络恢复后它就再也起不来了。代价是要配合**次数/时长告警**，不然"一直连不上"这件事没人知道。

条目三：设备与平台运维（5 题）

> **简历原话** 设备与平台运维：读取 Tegra 系统接口采集 GPU 利用率、显存与温度并周期上报，支撑边缘设备健康度监控；独立开发 GUI 批量部署工具（PyQt5），支持多路口配置批量生成与远程下发：Excel 工参经映射规则生成配置，经 SSH/SFTP 并行下发并软重启生效；耗时操作置于线程池、结果经信号槽回传主线程刷新表格，避免界面卡顿；工具已迭代至 V3 并由我独立维护升级。
>
> ⚠️ **本条是两块拼起来的**：**硬件监控**（参与）+ **部署工具**（我独立做的）。部署工具要重点准备——它是唯一能体现"能独立完成一个东西"的部分。
>
> 🔴 **本条两条红线**
>
> 1. **GPU 数据没有落库**：采集完直接上报，别说是"存进数据库再做字段比对"。
> 2. **主动说用了 AI 辅助**（部署工具），别等被问出来。

技术点 1：硬件指标采集（1 题）

Q1-1 你是怎么拿到 GPU 这些硬件指标的? [必背]

答：没有调任何厂商的私有接口，就是**读文件**。Linux 会把硬件的状态以**文件**的形式暴露在一个特定的系统目录下，所以想知道 GPU 忙不忙、显存占多少、温度多少度，直接打开对应的文件读出来就行，不需要接任何 SDK。

追问 1：为什么读文件就能拿到硬件信息？原理是什么？

> 这是 Linux 的设计哲学——**一切皆文件**。内核把硬件状态以伪文件的形式挂在一个系统目录下，读这个文件本质上是**触发一次内核查询并返回结果**，不是真的在读磁盘。所以开销极小，随便读。

追问 2：一共采哪几个指标？读法一样吗？

> 四个：**利用率、显存占用、显存占用率、温度**，读法各不相同。有的是一次能读出全部字段，有的要按行解析，有的本身就是个整型值。

追问 3：单位换算上有什么坑？

> 有两个地方必须换算，不做的话报出来的数是荒谬的：一是**字节到 MB** 要除两次 1024，少除一次就是一千倍的误差；二是**温度**有的接口给的是毫摄氏度，得再除 1000。这种错误不会崩，只会让监控图看着很离谱。

追问 4：为什么不直接调厂商的 SDK？读文件会不会不稳定？

> 调 SDK 要引入一整套依赖，还得跟着厂商版本走，边缘设备上多一个依赖就多一分风险。读文件的好处是**零依赖、跨版本稳定**——内核暴露的接口不会随便变。稳定性上做过长时间验证，没出问题。

追问 5：采集完之后数据去哪了？落库了吗？

> **没有落库**，采集完直接走上报。这里要说清：我负责的是"采得到、报得出去"，**后面平台怎么存、怎么展示不是我做的。**

追问 6：如果某个指标读不到怎么办？

> 读不到就**跳过这一个、不影响其他指标**，同时打一条日志。不能因为温度读不到就把利用率也丢了——监控最怕的是"整条数据断了"，而不是"少一个字段"。

技术点 3：部署工具整体设计（2 题）

Q3-1 这个部署工具是干什么的？为什么要做它？ [必背] ★ 最重要的开场题

答： 现场有大量路侧设备要配参数、升版本、查状态。原来的做法是运维拿着一份 Excel **一台一台 SSH 上去敲命令**——慢，还特别容易漏，一个路口配错了可能很久才发现。 所以我做了这款 Windows 桌面工具，把「看参数表 → 生成配置 → 远程下发 → 看结果」集中到一个界面。一句话说它的价值：**把"一台一台敲"变成"选表 + 点按钮 + 导出结果"。**

追问 1：为什么做成桌面工具，不做成网页？

> 现场同事在办公电脑上用，桌面工具能**直接读本地的 Excel**，也不用额外部署服务器；而且他们拿到的就是一个 exe，**双击就能跑**，不用装环境、不用配浏览器。

追问 2：配置是怎么从 Excel 变成设备上的配置文件的？

> 中间有一层**映射规则**：Excel 的列名 → 配置项的路径。我改的是映射层那几个文件，主流程完全不用动。这样现场说要加两个字段，我只改映射，不用碰生成逻辑。

追问 3：你怎么保证生成出来的配置是对的？

> 两层。一是**生成时校验**：必填项、取值范围、格式先卡一遍，生成不过就不出文件；二是**下发后比对**：把设备上实际在用的配置读回来跟本地这份对比，不一致会把具体差异项打出来——这才是真正能证明"生效了"的手段。

追问 4：是怎么连到那些设备上去的？

> 用 Python 的 SSH 库连上去，文件传输走另一条通道。连接本身做了超时和重试——现场网络环境差，没有超时的话一个连不上的设备能把整批卡死。

追问 5：几十台一起操作，界面会不会卡死？

> 不会。耗时操作都丢进**线程池**，结果通过信号回传主线程刷新表格。界面线程永远只做刷新，不干重活，所以几百台下去界面照样能拖能点。 👉 深挖见**条目三 Q4-1**。

追问 6：这个工具是你独立做的吗？用了 AI 辅助吗？

> 工具是我**独立开发并维护升级**的，已经迭代到 V3。开发过程中**用了 AI 辅助**——主要是搭界面和写重复的胶水代码，业务逻辑和现场适配是我自己做的。 👉 完整版见**条目三 Q3-8。主动说，别等被问出来。**

Q3-8 这个工具是你独立做的吗？用了 AI 辅助吗？ [保底] ★★ 诚实题，必答

答（照实说，这题诚实比包装值钱）： 工具是我**独立开发并持续维护**的，从第一版迭代到第三版。开发过程中我大量使用了 AI 辅助。 说清楚比较准确：**需求、架构、字段映射规则是我定的**，AI 帮我快速产出界面的骨架和一批样板代码，我再**逐行读、改、调**，最后自己打包分发给现场，后续迭代和维护也都是我在做。 第三版主要解决三个现场反馈的问题：域 ID 和订阅主题改成界面可配；证书目录没写权限导致上传失败；把配置的生成和部署合并成一个按钮，避免有人忘记点其中一步。 能佐证的是：它有完整的开发维护记录、有说明文档、有给现场用的操作手册，产物是现场正在用的东西。

> 💡 **说"用了 AI"不减分，说不清自己贡献了什么才减分。** 主动给出"规则我定 / 代码我读我改 / 迭代我维护"这三点，反而说明你能把 AI 用在刀刃上。

技术点 4：并发与界面不卡（1 题）

Q4-1 几十台设备一起操作，界面会不会卡死？ [必背] ★★ 高频

答： 会卡——所以**所有耗时的操作一律不放在界面线程。** 具体做法是：点按钮之后，先读取参数表把每台设备的连接信息和要传的东西组装好，然后按一个并发上限**分批**，**每一台设备一个后台任务**，投进线程池里去跑。后台任务连上去、传文件、执行命令，干完之后通过一个**信号**把结果对象传回界面线程，界面线程只负责更新表格和写日志，不做任何耗时的事。

技术点 5：远程下发与现场坑（1 题）

Q5-3 你们传文件遇到过权限问题吗？怎么解决的？ [加分] ★★ 最有含金量的一个坑

答： 遇到过。现象是目标目录属主是 root、我用的普通账号没写权限，传输直接报**权限不足。** 一开始我判断的是"目录在不在，不在就创建"，很快就发现不对：目录**明明存在**，就是写不进去。所以我把判据改成了**"当前用户能不能写"——这才是真正的成功判据。 解决套路是分级的：可写就直接过；不可写先试普通用户创建；还不行就走**提权**——用 `sudo` 把目录建出来、再把属主改成当前用户，之后同一账号走 SFTP 就能写了。密码走标准输入，**不拼进命令行**（避免转义，也避免被 `ps` 看到）。 还有个坑：部分设备的 sudo 要求必须有终端，第一次提权会失败，所以失败之后**自动带着终端重试一次。**好处是：现场不用在每台设备上手工配免密 sudo，也不用改成 root 登录。

条目四：交通参数检测服务（9 题）

> **简历原话** 4. **检测器框架与数据管道**：负责车头时距、流量检测器的实现，参与抽象基类 + 工厂模式的检测器框架搭建，各类复用同一套「逐帧累积、到点结算」多态流程；并以生产者-消费者模型实现数据链路——上游目标流经 DDS 订阅后写入有界阻塞队列，由独立工作线程消费，数据接收与检测计算解耦。 5. **车辆过线判定**：实现基于停止线几何相交的车辆过线判定，并叠加质量门控（位移阈值 + 横穿方向约束）过滤目标抖动与擦边误判，支撑流量与车头时距统计。
>
> 🔴 **本条三条红线**
>
> 1. **输入是结构化目标流，从没见过像素。**解码、检测跟踪、融合是算法团队的活。
> 2. **通信是 Fast-DDS，不是 CyberRT**（跟设备接入那块是两套中间件）。
> 3. **"上报"不是我做的**：口播讲到"独立工作线程消费、逐帧累积、到点结算"为止。
>
> ⚠️ **三条诚实口径**
>
> 1. 检测器实际是 **6 类**（还有排队 / 车速 / 过街 / 溢流），**没有逆行**；简历收敛写成"负责车头时距、流量检测器"。
> 2. **质量门控只对行人和非机动车严格生效，机动车不拦**（详见 Q3-5）。"支撑流量与车头时距统计"站得住，但**绝不能说成"所有过线目标都过了这道门"**。
> 3. "**负责**…检测器的实现"和"**参与**…框架搭建"是**有意分开**的，别说反、别升格。

技术点 1：检测器框架与多态（3 题）

Q1-1 这个交通参数检测服务，整体上是怎么设计的？ [必背]

答： 它跑在边缘设备上，形态是一个**算法插件**。上游融合感知把结构化的目标流推给我，我订阅收下写进一个**阻塞队列**，由一个工作线程取出来消费——每来一帧就广播给一组检测器各自累加，到配置的周期才统一结算出数。 检测器我用**抽象基类 + 工厂**统一管理，主循环只认基类、不关心具体是哪一类。 一句话：**订阅 → 排队 → 取帧 → 各检测器累加 → 到点结算**，是一条流水线。

追问 1：算法插件是什么意思？它跟宿主程序怎么配合？

> 插件本身就是一个**动态库**，宿主程序启动时按约定的一套 C 接口把它加载起来，我只要把那几个约定的回调实现好就行——初始化、给我数据、取结果。好处是算法和宿主彻底解耦，换算法、升级算法都不用重新编译宿主。

追问 2：你说的"结构化目标流"到底是什么？是图像吗？

> **不是图像，是一帧里所有目标的清单**——每条记录带 id、车型、经纬度、速度、车道号。图像解码、目标检测跟踪、多传感器融合那一整套是算法团队做的，我拿到的是他们的结果，**从头到尾没见过像素。**

追问 3：那你是怎么订阅的？收到的是原始数据还是解析好的？

> 订阅通道是**宿主框架那边声明的**，报文到了会通过回调递给我，我拿到的是一段 **JSON 文本**。我在回调里做的唯一一件事就是把它**反序列化成一帧结构化的目标列表**，然后立刻入队返回——**这一步只做解析，不做任何计算**。

追问 4：这个阻塞队列是你自己写的吗？怎么实现的？

> **不是，是工程里既有的一个组件**，我拿来用，没自己造。它内部是**互斥锁 + 条件变量**，底层是个双端队列，还带**容量上限**。我做的是在它上面搭这条生产-消费链路，轮子本身不是我的。

追问 5：为什么非要"阻塞"？队列空的时候那个线程在干嘛？

> 空的时候就**挂起睡过去**，有新数据入队时被唤醒，一点 CPU 都不占。如果写成轮询，线程就得一直转着圈查"有没有、有没有"——边缘设备上这点算力很金贵，不能这么烧。

追问 6：那为什么不直接在订阅回调里算完？非要多这一跳？

> 因为**回调里做重计算会堵住接收**。感知数据是持续来的，回调线程一旦被计算占住，后面的帧就收不进来，直接丢帧。加队列之后"收"和"算"拆成两段，各自按自己的节奏跑，互不拖累——**这就是简历上说的"数据接收与检测计算解耦"**。

追问 7：为什么只有一个工作线程？不怕算不过来吗？

> 当前的数据量下一个线程够用。而且单消费者有个很大的好处：**检测器内部那些累加账本不用加锁**——同一时刻只有这一个线程在改它们，天然线程安全。要改成多消费者并行，就得给每本账加锁，复杂度和出错概率都会上去。

追问 8：你说的"一组检测器"是多少个？它们之间什么关系？

> 准确说是一个**输出通道**：配置里给每个通道写清楚"多久出一次数"和"这一次要哪几类指标"，通道下面再**按车道、按断面各实例化一个检测器**。所以同一类指标可以同时挂在好几个通道上——比如流量既出秒级的、也出 5 分钟的，**那是两本账，各攒各的，互不干扰**。

追问 9：周期是怎么定的？到点之后算出来的数给谁了？

> 周期写在**配置文件**里，每个通道各配各的。"到没到点"的判定是框架既有的能力，我参与的是让服务支持可配置周期——**别说成"这个周期机制是我从零设计的"**。到点出数之后我调的是**宿主框架的发布回调**，把结果报文交出去就完事；后面发到哪个通道、平台怎么收，**不是我做的那一段**。

> 📍 **这条流水线一定会被拆开问**——简历第 4 条里每个词，对应后面哪一题：
>
> - **"DDS 订阅" / "独立工作线程消费"** → 技术点 2 的 Q2-1、Q2-4
> - **"有界阻塞队列"** → Q2-2（隔开的好处）、Q2-3（满了会丢）
> - **"生产者-消费者模型" / "数据接收与检测计算解耦"** → Q2-1 追问、Q2-4
> - **"抽象基类 + 工厂模式"** → Q1-2、Q1-3
> - **"各类复用同一套...多态流程"** → Q1-4
> - **"逐帧累积、到点结算"** → Q1-5
>
> 背法：Q1-1 是**目录**，后面 22 题是**展开**。面试官从你这句话里随便挑一个词往下问，你都能对上号——**答完 Q1-1 别停，等他挑词。**

> 📌 *自己复习：链路走法——宿主回调 → 解析 JSON 成一帧 → `put()` 入队 → 工作线程里 `take()` → 双重 for 广播 `update()` → 判周期 → `calculate_once()` 出数 → 发布回调。队列是 `afl::BlockingQueue`（muduo 风格，作者 Shuo Chen，BSD 协议），容量非 DEBUG 时 60，用到 80% 会打 warning。*

Q1-5 简历里写的「逐帧累积、到点结算」具体是什么意思？ [必背] ★★ 核心

答： 这是消费一帧的**两拍**。第一拍是**逐帧累积**：每来一帧就把它广播给所有检测器，各检测器只往自己的账本里累加，**这一步不出数**。 第二拍是**到点结算**：用帧时间戳判断有没有跨过配置的周期边界，跨过了才让每个检测器把账本算成结果、并且**算完就清零**，没跨过就直接取下一帧。 分开的好处是统计窗口干净——算完即清零，相邻两个周期不会重叠、也不会重复计数。

追问：那"周期"是从哪来的？

> 是**配置项**，配置里写的周期是多少就按多少聚合。触发逻辑是框架既有的能力，我参与的是让服务支持可配置周期——**别说成"这个周期机制是我从零设计的"**。

Q1-6 你们一共支持几类检测器？都算什么？ [必背] ★★ 诚实题

答： 代码里是 **6 类**：流量、排队、车速、车头时距、过街行人、还有溢流。简历上我收敛写成"负责车头时距、流量检测器的实现"，是因为这两类是我真正动手做的。 每类的产出不一样：流量出的是分车型的通过数和折算值；排队出的是排队长度和排队车辆数；车速出的是区间平均车速；车头时距出的是前后两车通过同一断面的时间间隔；过街是统计行人过街；溢流是判断路口会不会堵死。 说一下，**没有逆行检测**，别顺着面试官的话头承认。

追问：那几类的差别大吗？能统一到一套接口里？

> 差别主要在"吃进去之后怎么记、吐出来的时候怎么算"，以及**统计的空间粒度不一样**——有的按车道出、有的按整个断面出、有的按路口分支出。但这些差异都封在子类内部，主循环看到的还是那两个接口。

技术点 2：生产者-消费者数据管道（3 题）

Q2-1 上游的数据是怎么进到你这个模块的？ [必背]

答： 通过 **DDS 订阅**收下来的。上游融合感知把结果发布到一个约定的通道上，我这边订阅它，收到之后先把报文解析成结构化的一帧，然后**塞进一个阻塞队列**就立刻返回。 这里要注意：我们这块用的是 **Fast-DDS，不是 CyberRT**——跟设备接入那套是两套中间件，业务上靠标准化的感知消息衔接，代码上彼此不依赖。

追问：为什么不直接在收到数据的回调里算，非要加个队列？

> 因为**回调里做重计算会堵住接收**。感知数据是持续来的，回调线程一旦被计算占住，后面的帧就收不进来，直接丢帧。加队列之后"收"和"算"拆成两段，各自按自己的节奏跑，互不拖累——**这就是简历上说的"数据接收与检测计算解耦"**。

Q2-3 队列满了会怎样？会丢数据吗？ [必背] ★★ 容易答错

答： **会丢，这是有意的。** 容量满了之后，新来一帧就把**队列里最旧的那一帧挤掉**，给新的腾位置。 这是"有损但稳健"的取舍：边缘设备内存有限，与其让队列无限堆积把模块撑爆，不如丢掉最旧的数据保住稳定性——反正交通参数统计是持续采样，丢一帧老数据的代价远小于模块崩掉。 但有一点要分清：**已经被工作线程取走、正在处理的那一帧，绝不会被丢**，丢的只是还在排队等着的。

追问：那不就保证不了每帧必达了？

> 对，本来就**没承诺每帧必达**。这个场景要的是"持续稳定地产出统计结果"，不是"一帧都不能少"。如果真要求不丢，就得换成能持久化或者阻塞上游的方案，代价和复杂度都不是一个量级。

Q2-6 算出来的数最后是给谁了？上报是你做的吗？ [必背] ★★ 诚实题

答： 出数之后我调的是**宿主框架给的发布回调**，把结果报文交出去就完事了。 **后面发到哪个通道、平台侧怎么收、怎么展示，不是我做的那一段**，我不清楚细节，也不该硬编。 所以简历上我只写到"独立工作线程消费"为止——我能保证的是**到我这一层的输入和输出是对的**：目标流收得进来、指标算得出来、周期边界干净。

> 💡 这题答"不是我做的"不减分，硬编细节才减分。边界清楚本身就是加分项。

技术点 3：过线判定与防抖（3 题）

Q3-1 怎么判断一辆车真的过了停止线？ [必背] ★★ 核心

答： 记每辆车**上一帧的位置**，本帧到了就把"上一帧位置 → 当前帧位置"连成一条**位移线段**，看这条线段跟**虚拟停止线**相不相交。 相交只是必要条件，还得再过两道闸：位移要够大（不能是原地抖），方向要够"横穿"而不是沿着停止线擦过去，而且这辆车之前没被计过数。三条都满足才算真正过线。 停止线不是物理的线，是配置里给这条检测断面画的**一条虚拟线段**。

追问：为什么要用两帧连成线段？直接看当前帧位置不行吗？

> 不行，**只看单帧位置会漏掉高速车。**30 帧的视频两帧间隔三十几毫秒，快车一帧能挪出去一米多——很可能上一帧还在线这边、下一帧已经跳到线那边，两帧的位置都不在线上，单帧判断就认为它压根没过线。连成线段才能覆盖这种"跨帧跳过去"的情况。

Q3-2 怎么滤掉"停在停止线附近抖一下就算过线"？ [必背] ★

答： 两道闸。**第一道是位移阈值**：这一帧挪动的距离要超过一个下限（不到半米的不算），原地抖动直接忽略。 **第二道是方向**：位移方向跟停止线法线的夹角要够小，大约 60 度以内才认。那些**沿着停止线方向平移**的——比如车贴着线往前蹭——方向是平行的，不算过线。 第三还有一个记录：每辆车判过一次就记下来，**同一个统计周期内不再重复计。**

> 📌 *自己复习：简历上把这两道闸写作"质量门控（位移阈值 + 横穿方向约束）"——"横穿方向约束"就是上面第二道。代码常量在 `detectors/stop_line_crossing_helper.cpp`：`MIN_MOVE_DISTANCE = 0.5` m、`MIN_COS_TO_NORMAL = 0.5`（即 cos 60°）。口播只说"约半米""约 60 度"，不报常量名。*

Q3-5 你说的那两道防抖，对所有目标都生效吗？ [加分] ★★ 细节加分

答： 这里有个容易忽略的点：**只对行人和非机动车严格生效，机动车其实不拦。**代码里是分目标类型决定要不要走这个门控的。 原因也合理：机动车移动干脆、轨迹稳定，不需要卡这么严；但行人走两步位置就来回抖、还经常沿着停止线方向移动，不卡的话会大量误判。 所以简历第 5 条那句"支撑流量与车头时距统计"，要这么理解：门控是过线判定这一环上的**公共零件**，护住的是这两份统计里**行人 + 非机动车**那部分口径——流量本来就要出分车型明细，行人和非机动车就在明细里；车头时距也是**全类型都算**。 至于**机动车总流量**那一栏，机动车本来就不过这道门，所以它不受门控影响。 一句话：**这道门护的是行人 / 非机动车的过线质量，不是所有过线目标**——绝不能吹成"所有目标都过了质量门控"。

追问：那机动车不拦，会不会把贴着线擦过去的也算进来？

> 有可能，但概率低——机动车跟踪稳、轨迹连续，很少原地抖动。我的理解是这里有意只给行人 / 非机动车加严：**慢速蠕行往前挪的车，两帧位移也可能不足半米**，机动车要是也卡这道门，反而会把真实过线漏掉。而原地抖动和沿路方向平行移动这两类问题，恰恰主要出在行人和非机动车身上。

> 📌 *自己复习：`classify_volume_category()` 先定类型 → `stop_line_crossing_helper.cpp:60` 的 `strict` 只对 `PEDESTRIAN` / `NON_MOTOR` 置真，机动车走 `if(!strict) return true` 直接放行；流量输出 `kVolumeDetailKeys = Volume1..Volume5`（Volume1 行人 / Volume2 非机动车 / Volume3~5 机动车），`Volume` 机动车总流量只加 Volume3~5。*

附录：通用追问（4 题）

Q1 你在这个项目里具体是什么角色？哪些是别人做的？ [必背] ★★ 最重要的一题

答： 我是实习生，主要做了两块：**设备接入与数据链路**，以及**交通参数检测服务**。 具体的：**接口怎么抽象、工厂和自注册怎么设计、多路拉流这一块的实现和调试、GPU 指标的采集，还有交通参数这边的检测器框架、数据管道和车辆过线判定**，这几块是我在 mentor 指导下负责开发和调试的。整体架构是 mentor 定的，我负责把具体模块落地。 部署工具是我独立完成的。中间的解码、目标检测跟踪、多传感器融合那部分是算法团队做的，我不参与——我拿到的是他们的结构化结果。

> 💡 这题的打分点在于**边界清晰**。说"我参与了 X、Y，Z 不是我做的"，比含糊地说"这个项目我都接触过"靠谱得多。

Q2 你遇到过最难的调试是什么？怎么定位的？ [加分] ★

答（重点是"怎么查"而不是"多难"）： 印象最深的是加载失败那个。现象是某个设备类型在目标机器上就是注册不上，但**在我自己的开发机上是好的**——这种"环境相关的失败"最难查。 我的排查三步：先确认不是代码问题（同一份代码在开发机跑得通）；再缩小到环境差异（是不是依赖库缺失）；最后用查依赖的工具对比两台机器，发现目标机上少了一个间接依赖。解决办法是**把依赖检查前置**——加载之前先确认依赖都在。 教训是：**凡是"在某台机器上不行"的问题，八成是环境的锅，先把环境差异列出来比对，比闷头看代码有效得多。**

Q3 你的视频流用的是什么协议？为什么选它？ [必背]

答： 用的是 RTSP，实时流传输协议。选它主要有三个原因：第一它**支持双向控制**，除了拉流还可以发暂停、跳转这类控制指令；第二它**延迟比较低**，适合实时监控这种对时效敏感的场景；第三它是**安防监控领域的行业标准**，市面上绝大部分网络相机都原生支持，我们不用为了接某个厂商去做私有适配。 它的工作方式是：先通过协议完成握手和参数协商，然后视频数据才通过另一个通道持续过来，控制和数据是分开的。

追问：那跟 HTTP-FLV 有什么区别？

> HTTP-FLV 基于 HTTP，穿透性更好（走 80 端口基本不会被防火墙拦），但它只能单向拉流、没有控制能力，而且延迟相对高一些。路侧监控看重低延迟和控制能力，而且设备本身原生支持 RTSP，所以选它比较自然。

Q5 你觉得自己这个项目里有哪些做得不够好的地方？ [加分] ★ 必问收尾

答： 坦诚说有几点。第一**可观测性不够**：加载成功与否、注册了哪些设备、每路流当前健康状态，这些都应该在启动时一目了然地打出来，现在是出事了才去翻日志。 第二**统计是全量累计而不是滑动窗口**：像帧间隔这种，跑久了早期数据会一直留着，反映不了"当下"的状态。 第三**缺少主动告警**：现在基本都是"采集了、记录

了、等人来看"，理想情况应该是超阈值就主动推送。 第四**测试不足**：当时的验证主要靠实际部署环境和手工构造场景，**没有系统性的单元测试**——有些边界条件如果当时有单测，根本不会拖到现场才发现。

> 💡 主动承认问题 + 给出改进方案 = 有反思能力。比"我觉得挺好的"强一百倍。

最后两句

> 这份是**常用字表**，不是全集。遇到这里没有的问题，回 `0面试QA总表.md` 按题号查——题号是通的。
>
> **面试前一晚怎么过**：只背 `[必背]` 那批 + **六条总览题的追问链**（Q0.1、条目一 Q1-1、条目二 Q1-1 / Q4-1、条目三 Q1-1 / Q3-1、条目四 Q1-1）。把这六条链答顺了，面试官从你话里随便挑哪个词往下问，你都接得住。